

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



LẬP TRÌNH NHÂN LINUX PHẦN 2. TỔNG QUAN

Ngành: Công nghệ thông tin

Chuyên ngành: Kỹ thuật phần
mềm nhúng và di động

Mã số: 52.48.02.01

Nội dung

- Ưu điểm nhược điểm
- Tạo và hủy thread
- Đồng bộ giữa các thread

Định nghĩa

- Thread là một luồng các các lệnh mà cpu coi là một đối tượng khi sử dụng bộ lập lịch.
- Example: Chạy đồng thời scanf và printf hoặc scanf và write().

Ưu điểm nhược điểm

- Tối ưu được hiệu năng xử lý của tất cả các core
- Các thread được lập lịch một cách độc lập
- Nếu 1 thread gặp lỗi, tất cả các thread còn lại sẽ bị terminate.
- Tránh được trạng thái block chương trình
- Cần lưu ý đồng bộ dữ liệu giữa các thread
- Mỗi khi trong source code xuất hiện những đoạn⁴ code cần thời gian xử lý lâu và có thể chạy độc lập với đoạn code bên dưới thì nên nghĩ đến việc tạo thread mới để xử lý.

Tạo mới một thread

- `#include <pthread.h>`
- `int pthread_create(pthread_t *restrict tidp, const pthread_attr_t *restrict attr, void *(*start_rtn)(void *), void *restrict arg);`
- Returns: 0 if OK, error number on failure

Kết thúc một thread

- Thread call `pthread_exit`, return hoặc bị cancel bởi 1 thread khác.
 - `void pthread_exit(void *rval_ptr)`
- 1 thread khác phải clear resource của thread đã exit
 - `int pthread_cancel(pthread_t tid);`
- `int pthread_join(pthread_t thread, void **rval_ptr)`
 - Block cho đến khi thread cần đợi kết thúc
 - Return `EINVAL` nếu thread kia đã kết thúc từ trước

6

Thực hành

- Đếm số lượng số lẻ từ 1 đến 100 tỷ. Code theo 2 trường hợp, trường hợp 1 sử dụng multithread, trường hợp 2 không sử dụng multithread. So sánh thời gian của 2 trường hợp.
- Viết 1 chương trình C tạo 10 file, mỗi file chứa 5 triệu số ngẫu nhiên có 1 chữ số. Sử dụng multithread và không multithread rồi so sánh thời gian.

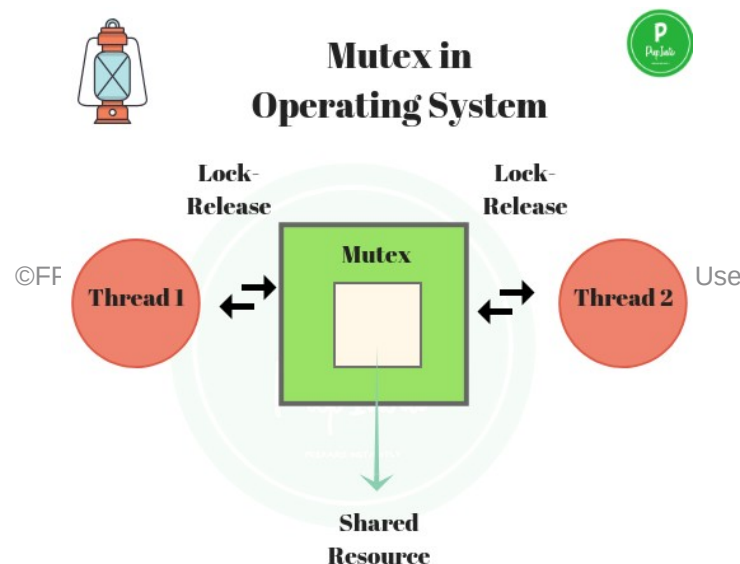
7

Mất đồng bộ dữ liệu giữa các thread

- Ví dụ minh họa về mất đồng bộ khi 2 thread cùng đọc ghi vào 1 biến
- Mất đồng bộ ghi đọc ghi vào mảng và file

Mutex

- Trong Linux, process sẽ rơi vào trạng thái sleep mỗi khi tiến hành lock mutex
- Chỉ có một khóa duy nhất.
- Trong một thời điểm chỉ 1 thread có được khóa



Mutex

- Function:
 - `int pthread_mutex_init(pthread_mutex_t *restrict mutex, const pthread_mutexattr_t *restrict attr)`
 - `int pthread_mutex_destroy(pthread_mutex_t *mutex)`
 - `int pthread_mutex_lock(pthread_mutex_t *mutex)`
 - `int pthread_mutex_unlock(pthread_mutex_t *mutex)`
 - `int pthread_mutex_trylock(pthread_mutex_t *mutex);`
- Tham khảo thêm
 - ✓ <https://vimentor.com/vi/lesson/thread-synchronization>

10

Thực hành

- Viết chương trình C tạo 2 thread, cả 2 thread lần lượt tăng biến đó lên 1 đơn vị, sử dụng mutex để đồng bộ việc ghi vào biến đó.
- Viết 1 chương trình C tạo ra 2 thread, 2 thread đều ghi vào chung 1 file, sử dụng mutex để bảo vệ file, nên sử dụng wrapper function thay cho hàm ghi file thông thường.

11

Dead lock

- Là trạng thái trong hệ thống có 2 hoặc nhiều luồng xử lý đi vào vòng lặp vô tận để chờ đợi tài nguyên của nhau.
- Có khả năng xuất hiện khi hệ thống có 2 hoặc nhiều thread sử dụng chung nhiều lock để hoàn thành công việc.
- Cách khắc phục: Giảm số lượng lock trong source code.

12

Semaphore

- Có 1 hoặc nhiều khóa
- Trong một thời điểm có thể có 1 hoặc nhiều thread có được khóa

Semaphore

- Source code
 - `sem_t sem_name;`
 - `int sem_init(sem_t *sem, int pshared, unsigned int value);`
 - `int sem_wait(sem_t *sem)`
 - `int sem_post(sem_t *sem);`
 - `int sem_destroy(sem_t *sem);`

14

Wrapper function

- Tạo các hàm đọc ghi riêng cho những tài nguyên có thể bị xung đột.
- Sử dụng các trick của C để giấu đi tài nguyên đó, chỉ cung cấp ra ngoài các hàm đọc ghi.
- Minh họa

15

Thực hành

- Viết 1 chương trình C tạo ra 3 thread, các thread lần lượt được tăng 1 biến chung thêm một đơn vị và ghi giá trị mới vào 1 trong 2 file output. Sử dụng semaphore để đồng bộ việc ghi vào 2 file
- Tạo 10 ma trận kích thước 200x200. Nhân 10 ma trận đó với nhau sử dụng multi thread.

HỎI - ĐÁP
